Increasing Network Security with eBPF

Assignment #1 - State of the art

Faculty of Sciences · University of Porto

António Lanção (up202205416)
David Sá (up202303580)
Henrique Teixeira (up202306640)

Network Security 2026/27
01 November 2026

Abstract. Extended Berkeley Packet Filter (eBPF) lets verified programs run at hook points spanning the whole kernel, from packet reception (XDP) to permission checks (BPF LSM), with maps as shared state. This makes it a substrate for security systems rather than a security product, and two questions follow. First, which hook should a defense use: we compare hook families on packet state, verdicts and cost, and derive two use cases from them. Second, is the substrate itself trustworthy: we survey the execution and safety model, the verifier, CO-RE portability, and the privilege boundary, and build a threat model for eBPF deployments from published cross-container attacks and verifier critiques. Defense systems are then compared as architectures: production (Cilium, Tetragon) against research (bpfbox, BPFContain), finding that policy authorship, not capability, separates them. We close with a prototype proposal for Assignment #2, a two-layer firewall (XDP for packets, cgroup/socket hooks for workloads), with evaluation criteria tying enforcement correctness and cost against nftables to the least-privilege deployment the threat model demands, and an implementation roadmap.

Table of Contents

1 Introduction 4
2 The eBPF execution and safety model 5
2.1 Program lifecycle: bytecode, loading and JIT 5
2.2 The verifier 5
2.3 Maps, ring buffers and helpers 6
2.4 Portability: BTF and CO-RE 6
2.5 Verifier safety vs. authorization to load 6
3 Hook points for security 8
3.1 Overview of hooks 8
3.2 Use case A: volumetric / DDoS filtering 9
3.3 Use case B: container-aware connection control 9
4 eBPF security systems 10
4.1 Production: Cilium and Tetragon 10
4.2 Research: bpfbox and BPFContain 10
4.3 Comparison 11
5 Threat model for eBPF itself 12
5.1 Assets, trust boundaries and assumptions 12
5.2 Adversaries and attacks 12
5.3 Mitigations 12
6 Limitations and open challenges 14
7 Proposed prototype and evaluation criteria 15
7.1 Scope 15
7.2 Metrics 16
7.3 Environment 16
8 Ethical and legal considerations 17
9 Conclusion 17
10 Implementation roadmap for Assignment #2 18
Bibliography 19

List of Figures

Figure 1 Where each eBPF hook sits in the kernel. The receive path runs bottom to top; `sk_buff` allocation separates XDP from every later network hook. kprobes and tracepoints can observe any layer but cannot return a verdict. 8
Figure 2 Prototype flow. Inbound packets meet the XDP program in the driver and are dropped or passed before an `sk_buff` exists; container `connect()`/`bind()` calls meet the cgroup hooks. Both read policy from pinned maps that the controller updates at runtime and that feed counters and the drop log back to user space. In baseline runs the XDP program is detached and nftables applies the same rules in the stack. 16

List of Tables

Table 1 Security-relevant eBPF hooks. 9
Table 2 Production vs. research eBPF security systems. 11
Table 3 Evaluation criteria for Assignment #2. 16
Table 4 Milestones, dependencies and validation. Weeks count from 2 November 2026 to the Assignment #2 deadline of 6 December 2026. Week 5 (from 30 November) is left free for the Assignment #2 report and packaging. 18

List of Acronyms

eBPF	**Extended Berkeley Packet Filter**
sk_buff/skb	Socket Buffer
NIC	Network Interface Controller
XDP	eXpress Data Path
tc	Traffic Control
LSM	Linux Security Module
JIT	Just-In-Time compilation
BTF	BPF Type Format
CO-RE	Compile Once, Run Everywhere
TCB	Trusted Computing Base
TOCTOU	Time-Of-Check to Time-Of-Use
LPM	Longest Prefix Match
CIDR	Classless Inter-Domain Routing
DDoS	Distributed Denial of Service
YAML	YAML Ain't a Markup Language
FPR	False Positive Rate
QEMU	Quick Emulator
KVM	Kernel-based Virtual Machine
LTS	Long-Term Support

1 Introduction

Classic kernel extension mechanisms force a choice: loadable kernel modules run arbitrary code with full kernel privilege and must match the running kernel exactly, while fixed in-kernel subsystems such as netfilter or the LSM framework only implement the policies their authors anticipated. Extended Berkeley Packet Filter (eBPF) removes the choice by letting untrusted code run **inside** the kernel safely: programs are compiled to a small instruction set, statically proven safe by an in-kernel verifier before they ever run, JIT-compiled to native code, and attached to hook points that span the whole range of kernel work, from the earliest moment a packet exists (XDP, in the driver, before any allocation) to the kernel's own permission checks (BPF LSM), with maps as shared state between kernel programs and user space [1], [2], [3].

For network security this makes eBPF a **substrate** rather than a product: it provides safe programmability at kernel hook points and defense systems are built on top of it as architectures of programs, maps and user-space control planes. Two things follow and both structure this report:

- First, the choice of hook is a design decision with security consequences: each position in the kernel trades visibility for cost, so the same attack is defensible at different prices depending on where the defense sits.
- Second, the substrate is itself security-sensitive: the verifier is a large, hand-written component of the Trusted Computing Base whose bugs are kernel bugs [4], and the systems built on eBPF add their own seams; cross-container attacks that confuse the identity mapping or exploit the policy data path show the confinement breaking at the joins, not the machinery [5].

This report compares the state of the art along those lines. Its contributions:

- a description of the eBPF execution and safety model, the load → verify → JIT → attach pipeline, the verifier's three families of checks and maps as the only durable memory, separating what the machinery guarantees from what it does not;
- a hook-by-hook comparison of the security-relevant attachment points (XDP, tc, cgroup/socket, tracepoints/kprobes, BPF LSM), instantiated with two use cases that lead to different hooks from the same question;
- a comparison of production systems (Cilium, Tetragon) with research confinement systems (bpfbox, BPFContain), showing each as a point in the same design space of hooks × programs × maps × control plane;
- a threat model for eBPF itself, assembling the verifier-as-TCB critique and the cross-container attack literature into adversaries and mitigations, which the prototype of Assignment #2 will partially validate.

The remainder follows that order: §2 the execution and safety model, §3 hook points and use cases, §4 security systems, §5 the threat model against eBPF, §6 limitations and open challenges, §7 the proposed prototype and its evaluation criteria, §8 ethical and legal considerations, §9 the conclusion and §10 the implementation roadmap for Assignment #2.

2 The eBPF execution and safety model

2.1 Program lifecycle: bytecode, loading and JIT

eBPF programs are compiled from restricted C into bytecode for a small register-based virtual machine: eleven 64-bit registers that map one-to-one onto hardware registers on the supported 64-bit architectures, a fixed instruction set and a single calling convention shared with the C ABI, which is what makes just-in-time translation to native code a faithful, mechanical restatement rather than a second implementation [1], [2].

Loading goes through a single entry point, the `bpf()` system call. The kernel takes the bytecode, hands it to the verifier (Section 2.2) and only after the proof succeeds does it pass to the JIT compiler, which translates each instruction to a small block of native machine code with no runtime checks, interpreter, or sandbox left in the hot path. The proven bytecode is the safety artifact; the JIT preserves its properties and invents no behavior of its own. The result is native-speed execution with zero per-event safety overhead: verification is paid once at load time, execution is fast forever.

A program only does something once it is **attached** to a hook point: the kernel invokes it as an event handler whenever the hooked event fires: a packet arriving at a driver, a syscall, a security decision. Loaded programs and their maps are reference-counted by the kernel and can be pinned to a filesystem path (bpffs) so they outlive the loader process [6].

2.2 The verifier

eBPF code runs in kernel address space, where a single bad memory access corrupts the machine every process depends on. Since the requirement is certainty about all executions, including inputs an attacker crafts, before the program ever runs, runtime mechanisms are categorically insufficient: a sandbox needs a floor below the kernel and there is none. Testing only covers the inputs somebody thought of. The verifier therefore **proves** safety by static analysis over all reachable paths, in two steps: a check that the control-flow graph is a directed acyclic graph, then a simulation of every instruction along all paths, with state pruning where an already-verified state covers a new branch [2], [7].

The proof tracks an abstract state: register types, pointer offsets and possible value ranges, through every path, enforcing three families of checks:

- **Termination**: General termination is undecidable, so the verifier restricts the class of allowed programs instead: historically it required an acyclic control-flow graph (no loops at all), and modern kernels admit bounded loops within an instruction budget. The check is mechanical precisely because the program class is restricted.
- **Memory safety**: Every load or store must be provably in bounds on every path: packet accesses against the `data_end` pointer in the context, map values against the map's declared value size, stack accesses against ranges previously written. Bounds checks in the program (e.g. `if (ptr + offset > data_end)`) are not a courtesy: they are how the programmer hands the verifier the proof and the verifier refines ranges at every branch.
- **Type safety**: Each register carries a type alongside its range and every instruction and helper call must be type-compatible: a packet pointer cannot be used as a map key, a map lookup's return must be checked for NULL before dereference. Type confusion is the classic kernel exploitation primitive and the verifier refuses to let a program create one [2].

The verifier does **not** check that the program does what its author intended, nor that it is efficient; its sole purpose is protecting the kernel's internals [2]. This makes it the trusted computing base (TCB) of every eBPF deployment: the component every other safety property depends on and which must itself be trusted rather than verified. The analysis is also an over-approximation: at merge points, states are unioned and precision is lost, so correct programs can be rejected, the practical cost of which grows with program complexity, as Jia et al. argue [4]. Improving the verifier itself is the community's main response: PREVAIL [8] reimplements the analysis in user space as an abstract interpretation, alongside fuzzing and formal verification of the existing verifier and JIT [4].

2.3 Maps, ring buffers and helpers

An eBPF program is an event handler: it is invoked, runs and returns with no state that survives between invocations. Since nearly every security task needs memory across events (a rate limiter needs per-source counters, a policy needs its rule set), state must live outside the program and that object is the **map**: a typed key → value store allocated by the kernel, read and written by programs through helper calls and equally reachable from user space [1], [2]. Maps thus serve three roles at once: persistent storage between invocations, coordination between programs attached at different hooks and the kernel ↔ user-space channel that a control plane reads and updates.

The map type encodes the access pattern. Security-relevant ones include hash maps (key → value lookup, e.g. per-IP counters), arrays (index → value), LPM tries (longest-prefix match, the data structure behind CIDR-based policy), per-CPU variants (each CPU has its own copy, so hot-path updates never take a lock), and ring buffers, which stream events to user space through a single buffer shared by all CPUs, with reserve/commit records, replacing the older per-CPU perf buffer [9]. Helpers and kfuncs are the kernel-provided interface a program calls to do anything beyond pure computation: map lookup, packet manipulation, getting the current process and cgroup identity, redirecting a packet. The verifier checks every helper call's argument types at load time, which is what keeps the interface safe despite the kernel backing it.

Maps are also where **policy** lives. A confinement system's LSM programs are generic; which container may open which file is data in a map the trusted user-space agent loads. This split (enforcement logic in the kernel, policy as data) is the pattern every system in Section 4 follows and also the attack surface He et al. exploit [5].

2.4 Portability: BTF and CO-RE

An eBPF program compiled against one kernel's structures will not load on another: offsets of fields in `struct sock`, or even the presence of a field, differ across kernel versions and configurations. The classic answers, either compiling per kernel or requiring vendors to keep stable kernels, both break the "policy as data" deployment model of §2.3.

BPF CO-RE ("Compile Once, Run Everywhere") fixes this with a division of labour [1], [10]:

- **BTF** (BPF Type Format) is the compact type description the kernel ships alongside itself: for each structure, which fields exist at which offsets.
- The **compiler** leaves the structure accesses in the object file as **relocations** ("field X of type Y") instead of hard-coded offsets.
- **libbpf**, at load time, matches those relocations against the running kernel's BTF and patches in the right offsets; where a field genuinely does not exist, a well-named compatibility check (`bpf_core_field_exists`) lets the program take a fallback path.

The verifier's guarantees are unaffected: the program is patched **before** verification, so safety is not traded for portability. For security tooling this is what makes one shipped policy binary run across a fleet of heterogeneous kernels, the deployment assumption both Cilium and the research systems of Section 4 make.

2.5 Verifier safety vs. authorization to load

> **Key distinction required by the brief**
>
> The verifier answers *"is this program safe to run in the kernel?"*.
> The privilege model answers *"who may load and attach it?"*.
> A program the verifier accepts can still be malicious.

The privilege model is separate from and older than the verifier: it decides who may reach the `bpf()` system call's privileged commands at all.

- **Root as gatekeeper**: Historically, loading eBPF programs required root (`CAP_SYS_ADMIN`). The verifier prevents harm to the kernel, but it cannot judge intent, so it is left to the administrator to decide which programs may be loaded [2].
- **The modern split**: Since Linux 5.8, `CAP_BPF` separates BPF operations from the overloaded `CAP_SYS_ADMIN`, with `CAP_PERFMON` for tracing and `CAP_NET_ADMIN` for network-attached programs: narrow, purpose-built capabilities instead of "effectively root" [5], [11].
- **Unprivileged BPF**: The sysctl `kernel.unprivileged_bpf_disabled` controls whether a process without `CAP_SYS_ADMIN` or `CAP_BPF` may call `bpf()` at all (0 allowed, 1 disabled until reboot, 2 disabled but reversible by an admin). Kernels built with `BPF_UNPRIV_DEFAULT_OFF` default to 2 [12]. Classic BPF (socket filters, seccomp) uses its own interfaces and is not affected by this switch.
- **Delegation without privilege**: A container or service should be able to load **its own** programs without holding host capabilities. A BPF token enables this: a privileged process creates the token from a bpffs instance mounted with `delegate_cmds`, `delegate_maps`, `delegate_progs` and `delegate_attachs` options and hands its file descriptor to the container. `bpf()` calls carrying the token then succeed only for the delegated operations, and only with capabilities held in the container's own user namespace, not on the host [13].

The distinction matters for the threat model of section 5: every "who may load what" question is answered on this layer, not the verifier's.

3 Hook points for security

3.1 Overview of hooks

The same verified bytecode means different things depending on where it is attached: the hook determines what the program can see, what it can enforce and what each decision costs. XDP runs in the driver **before** the socket buffer (`sk_buff`/skb) exists, so the program sees the raw frame and parses headers itself and its drop is the cheapest decision the kernel can make about a packet, which is why it is the first-line defense against volumetric attacks [2], [6]. tc hooks sit after the skb is built, at ingress or egress, and read skb fields such as marks and priority [6], so decisions that depend on stack context (per-flow classification, rate limiting, egress filtering) belong there. Tracepoints and kprobes are not on the packet path at all: they attach to kernel execution and observe the arguments of kernel functions (stable pre-chosen events vs. near-arbitrary instructions, stability against reach). cgroup/socket hooks see operations of a workload's sockets. BPF LSM attaches to the kernel's permission checks: its return value is a verdict on an operation rather than on a packet (deny with an error such as `-EPERM`), which is what turns a host-security system from observation into enforcement [3]. Figure 1 places each hook on the receive path. The rule for choosing is derived from position, not speed: **attach where the information you need comes into existence.**

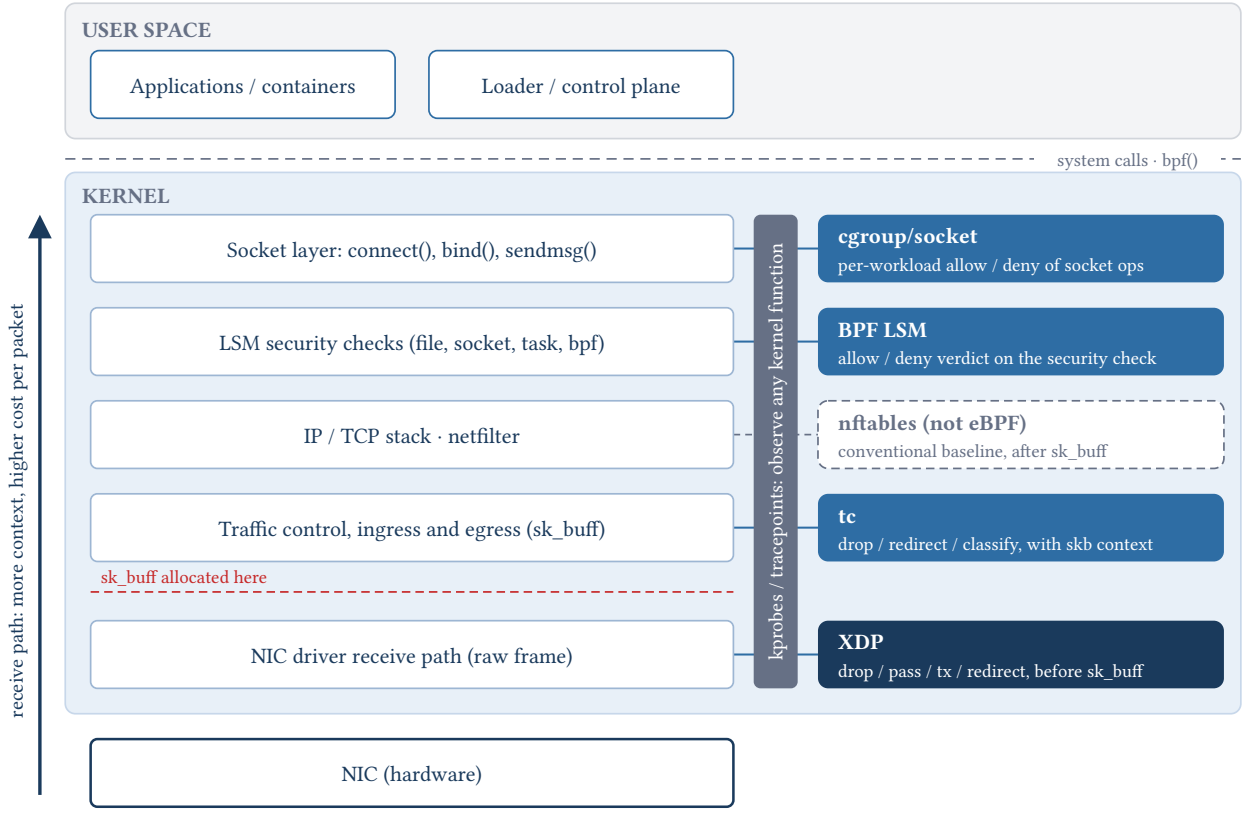


Figure 1. Where each eBPF hook sits in the kernel. The receive path runs bottom to top; `sk_buff` allocation separates XDP from every later network hook. kprobes and tracepoints can observe any layer but cannot return a verdict.

Table 1. Security-relevant eBPF hooks.

Hook	Context available	Enforcement actions	Overhead	Position
XDP	Raw frame, pre-skb	Drop, pass, TX (bounce out same NIC), redirect	Cheapest; line rate	ingress, in driver
tc	Post-skb, headers parsed	Drop, accept, redirect, classify	skb cost already paid	ingress/egress
cgroup/socket	Workload, socket	Deny or restrict socket ops	Per-socket, no skb cost	socket layer
kprobe/tracepoint	Kernel function args	Observe only	Low	anywhere
BPF LSM	Security check, credentials	Allow / deny	Return verdict on the LSM check; function-entry cost	LSM hooks

3.2 Use case A: volumetric / DDoS filtering

A server under a volumetric attack receives millions of junk packets per second, identifiable as garbage from header bytes alone. Header bytes are present in the raw frame, so identification does not require stack context and under attack the weapon is **cost per packet**, so the deciding factor is which drop is cheapest. XDP is the answer: the program parses IP/TCP headers itself and drops the flood before the kernel allocates a `sk_buff`, so the attack's packets never cost more than a few instructions each; in the XDP paper's DoS experiment, TCP throughput without the filter halves at 3 Mpps of attack traffic, while with the XDP filter it stays stable up to 19.5 Mpps, about 10 Gbps of minimum-size packets on a single core [2]. State the filter needs between packets (per-source counters, rate windows) lives in per-CPU maps, so the hot path never contends. tc **can** filter the same bytes, but each drop happens after the skb is already paid for: the wrong economics for a flood, the right ones once per-flow state or egress policy matters. The conventional baseline, nftables, sits in the stack after skb allocation. Comparing the two is the natural throughput measurement for an XDP filter, and the brief names it as the conventional baseline.

3.3 Use case B: container-aware connection control

Consider the same server, now running untrusted workloads in containers, with a policy requirement: **only the image-fetch service may open connections to the registry network; no container may bind the host's service ports**. This question is not about packets, but about **who is allowed to initiate what** and the identity "which container initiated this socket operation" does not exist at XDP's layer at all. The information comes into existence where the socket operation is performed, so the program attaches there: cgroup/socket hooks see the workload's `connect(2)`, `bind(2)` and friends with the cgroup context attached; then returns a verdict that restricts that workload's sockets (enforcement without parsing a single packet). State (which container may reach which network) lives in maps updated by the trusted user-space agent, shared with the enforcement program.

This mirrors use case A's shape (identify, decide in-kernel, keep state in maps), but the **question** chose a different hook, one layer away from the network. It is also BPFContain's starting point, one hook family further out: attach to the LSM checks that guard socket operations and the same policy now covers containers uniformly, with credentials in the context rather than in the socket [3], [14]. Both use cases together define the scope for Assignment #2 (Section 7): XDP answers "what may this packet do", the socket/LSM family answers "what may this workload do" and a secure system needs the answer to both.

4 eBPF security systems

4.1 Production: Cilium and Tetragon

Cilium applies the eBPF substrate to **networking policy**: instead of IP addresses and port rules, it derives a security identity for each pod from its labels, shared by pods with the same labels [15]. L3/L4 policy is enforced by eBPF programs where the traffic already flows (XDP or tc at ingress/egress, socket-level programs for per-workload rules) [6], while L7 (e.g. HTTP) rules are handed to an Envoy proxy in user space [16]. The same identity model thus drives routing, load balancing and policy in one place.

Tetragon, from the same ecosystem, targets **process and kernel behaviour** rather than the network: tracing policies attach kprobes, tracepoints and uprobes to security-relevant kernel functions, filter in the kernel on arguments and on Kubernetes namespaces and pod labels, and can enforce inline: overriding the return value so the call never runs, or sending a signal such as `SIGKILL` to the process [17]. Because a policy is data (a Kubernetes custom resource saying "watch these functions, filter on these arguments"), the in-kernel logic stays generic while the site-specific rules live in user space, the same split as §2.3′s maps.

4.2 Research: bpfbox and BPFContain

The production systems above ship policy for their own orchestration model. The research line asks whether eBPF can **replace** the patchwork of confinement mechanisms (namespaces, cgroups, seccomp, AppArmor/SELinux) with one coherent mechanism.

bpfbox is the proof of concept: per-process confinement policies that can restrict at the **userspace function**, **system call**, **LSM hook** and **kernelspace function** boundaries, a granularity no existing mechanism offers, in under 2000 lines of in-kernel code, using LSM programs for enforcement and uprobes, kprobes and scheduler tracepoints to track process state [18]. A policy can require, for example, that `execve` calls originate in a specific function of the confined binary, not just that certain binaries may be executed. In its web-server benchmark, alerting on every security operation costs about 8.4% throughput, slightly less than AppArmor in a comparable configuration, showing the generality is affordable.

BPFContain, by the same authors, generalizes bpfbox from processes to **containers**: a YAML policy language with default-deny and default-allow modes, semantically meaningful permissions on top of the raw LSM API and integration with existing container managers without modifying their source. Its prototype uses 38 LSM programs, two fentry programs, two scheduler tracepoints and one uprobe [14]. Its diagnosis is the inverse of Tetragon's opportunity: the kernel has no unified container abstraction, so isolation is today a mishmash of mechanisms whose combined policies are hard to audit and a failure in any one of them exposes the container. eBPF's LSM hooks close that semantic gap at runtime, with no kernel patches.

4.3 Comparison

Table 2. Production vs. research eBPF security systems.

System	Policy level	Hooks used	Identity	Reported overhead
Cilium	Labels → identity; L3/L4 in eBPF, L7 via Envoy	XDP, tc, socket	Workload labels shared kernel-wide	None published; claims in-kernel filtering keeps agent cost off the data plane
Tetragon	YAML tracing/ enforcement policies	kprobes, tracepoints	Process / container from events	None published; in-kernel filtering bounds per-event cost
bpfbox	Confinement policy, per-process	LSM + uprobes, kprobes, tracepoints	Process under confinement	≈8.4% web-server throughput; under 2000 lines in-kernel
BPFContain	YAML, container-level, default-deny/ allow	LSM + fentry, tracepoints, uprobe	Container	Under 16% across Phoronix benchmarks, on par with AppArmor

Overhead is directly comparable only in the research row: bpfbox and BPFContain publish benchmark methodologies and numbers (web-server throughput, Phoronix suites vs. AppArmor), while neither Cilium nor Tetragon publishes comparable figures, reporting only qualitative claims of low overhead from in-kernel filtering. This asymmetry is itself a finding, one the prototype of Assignment #2 will treat as a baseline to produce.

The two lines answer different questions. Production systems assume the orchestration model and buy into it (identity from labels, events from the runtime), trading generality for operational simplicity. Research systems start from the threat model: what a confined process should be able to do, expressed directly against kernel boundaries, at the cost of writing and auditing policy without a manager's help. The substrate is the same in both; what differs is who defines identity and how far below the API the policy reaches.

5 Threat model for eBPF itself

5.1 Assets, trust boundaries and assumptions

The assets are kernel integrity (memory, control flow, module state), the isolation between containers sharing a kernel, the integrity of policy and telemetry consumed by the systems of Section 4 and secrets that live in kernel memory or in other containers' address spaces.

The trust boundaries cross twice. In the loading path, untrusted bytecode crosses from user space into the kernel at the `bpf()` syscall: the verifier and JIT are the gate and everything they let through is trusted at runtime [4]. In the shared-kernel path there is no boundary at all for tracing programs: a program attached inside container A can read processes and memory of the host and of container B, because eBPF tracing is not restricted by UID or PID namespaces [5].

We assume the standard container protections are enabled (Seccomp, AppArmor/SELinux, KASLR, SMEP/SMAP, non-executable memory) and that the kernel itself has no exploitable memory-safety bug, following the threat model of [5]: the attack surface we model is eBPF's normal features, not a kernel exploit. We also assume the systems of Section 4 are deployed in good faith; a compromised or malicious agent is out of scope.

5.2 Adversaries and attacks

- **Malicious or compromised loader**: Whoever controls the loader can decide which programs are attached, but cannot make the verifier accept unsafe bytecode. The real risk is weaker: a loader with ambient `CAP_SYS_ADMIN`/`CAP_BPF` privilege that loads unsigned programs is a supply-chain choke point, since program provenance is not checked. He et al. note that eBPF malware can enter through poisoned container images [5], and Jia et al. propose signing extensions in a trusted toolchain and checking the signature at load time [4].
- **Over-privileged container**: A container granted eBPF capabilities can use tracing features (kprobes, raw tracepoints) to read any process's memory and opened files across containers (`bpf_probe_read_user`) and to kill any process (`bpf_send_signal`) with no kernel exploit at all [5]. The attacker needs either `CAP_SYS_ADMIN`, or `CAP_BPF` plus `CAP_PERFMON`; about 2.5% of the active Docker Hub repositories He et al. measured already grant the required permissions.
- **Unsafe helper or program type**: Helpers are unverified kernel functions that serve as escape hatches around the verifier's restrictions, and they also escape its guarantees: Jia et al. crashed the kernel from a verified program through a bug in `bpf_sys_bpf` (later a CVE) [4], and a fully-verified program that calls `bpf_probe_write_user` writes arbitrary process memory [5]. Similarly, map-tamper attacks use `bpf_map_get_fd_by_id` to reach another program's maps, paralyzing systems like Tetragon or Datadog by deleting their tail-call maps [5].
- **Verifier and JIT bugs as kernel attack surface**: The verifier itself is complex C code (thousands of lines, growing with every helper) and has had at least 22 bugs in two years [4]; a verifier bug (e.g. CVE-2022-23222, missing pointer validation) turns unprivileged bytecode into arbitrary kernel read/write and a JIT bug lets verified-safe code hijack kernel control flow.
- **Policy bypass**: Semantic gaps and races. He et al. show that all the existing LSM-based eBPF access-control tools they tested are vulnerable to **process identity confusion**: an attacker creates a process with the same path or PID as an allow-listed one inside the container and inherits its permissions [5]. More generally, a hook that fires on a decision may see different state than the decision will act on (TOCTOU between check and use); a rule written against one hook may miss the intent it was written for (the DDoS rule at XDP vs. the same intent at tc); and, under load, ring-buffer events can be dropped, so enforcement done in user space from those events can fail open.

5.3 Mitigations

Four layers, from broadest to narrowest. Disable unprivileged eBPF (`kernel.unprivileged_bpf_disabled`, which defaults to 2 on kernels built with `BPF_UNPRIV_DEFAULT_OFF`) so the syscall itself requires capability [12]. Restrict capabilities per container (deny `CAP_SYS_ADMIN`/`CAP_BPF`/`CAP_PERFMON` by default; a BPF

token delegates a narrow subset through a bpffs fd [13]) so a compromised container still cannot load. Constrain what loaded programs may do: He et al. propose CapBits, a permission model that gives each process an allow-list of program types, helpers and tracing scope and keeps it in kernel-held fields, which also resists the identity-confusion attack above [5]. bpfbox and BPFContain apply the same idea to their own state, with an LSM program on `bpf(2)` that stops other processes from touching their programs and maps [14], [18]. Audit provenance and behavior at runtime: sign or fingerprint programs at load and watch for the map-tamper pattern (`bpf_map_get_fd_by_id` against foreign IDs).

The prototype of Assignment #2 applies the first two: it runs with the minimum capability set (measured before and after) and on a kernel with unprivileged BPF disabled. Its threat-model audit (milestone M5) walks each attack above against the deployed configuration, including a map-tamper attempt against its own policy maps.

6 Limitations and open challenges

The sections above share one theme: eBPF's safety story is a verification story and verification guarantees less than it appears to.

The verifier is part of the TCB and a growing one: The verifier guarantees memory safety, termination and type safety for the bytecode it accepts [2], but it is ad-hoc C code that has grown with every helper and program type, has had at least 22 bugs in two years and admits escape hatches (helpers) that bypass the very guarantees it exists to provide [4]. The community responses under way, fuzzing, verifying the verifier, or proof-carrying code, all assume the verifier stays; Jia et al. argue the deeper fix is moving extensions to a safe language (Rust) with signed toolchains and lightweight runtime checks, eliminating the in-kernel verifier.

Helpers weaken the guarantee at the boundary: A verified program calling `bpf_probe_write_user` writes arbitrary process memory; one calling `bpf_map_get_fd_by_id` reaches another program's maps. The verifier cannot help, because the hazard is in what the helpers are **for** (tracing, inspection), not in what the bytecode does. This is why the container threat model of [5] exists at all: no kernel bug is needed, only a container with eBPF capability and a kernel that permits these helpers.

Observability and enforcement trade availability for precision: Hooks that merely observe (kprobes, tracepoints) are cheap and everywhere; hooks that enforce (XDP drop, BPF LSM verdict) stand in the data or execution path, where a false positive drops legitimate traffic or kills a legitimate process and a bug in the program is a kernel-space outage. Even inline enforcement has gaps: Tetragon warns that a `SIGKILL` alone does not guarantee that an in-progress operation such as a `write()` is stopped, and must be paired with a return-value override [17].

Portability is solved for structure, not for semantics: CO-RE handles layout differences across kernels [10], but a policy still behaves differently across hook ordering changes, LSM sets and kernel versions; "compile once, run everywhere" does not free the operator from testing per deployment kernel.

The permission model lags the attack surface: Capabilities gate the loading of programs, but once loaded, tracing programs see the whole kernel; the capability split of Linux 5.8 and the BPF token are coarse answers to a problem [5] show needs per-program, per-process confinement (which is the gap bpfbox and BPFContain address at the policy layer).

7 Proposed prototype and evaluation criteria

7.1 Scope

A **two-layer eBPF firewall** that turns the two use cases of §3 into one small, auditable component. Each layer answers one of the questions those use cases separated: XDP answers "what may this packet do", and the cgroup/socket layer answers "what may this workload do".

- **Ingress layer (XDP)**: a program in the driver receive path drops traffic before an `sk_buff` exists. It applies three rule classes held in maps: a CIDR blocklist in an LPM trie, a protocol/port denylist in a hash map, and a per-source packet-rate limit with per-CPU counters, so the hot path never takes a lock. Every verdict increments per-rule counters, and dropped packets are sampled into a ring buffer for logging.
- **Workload layer (cgroup/socket)**: `connect4`/`connect6` and `bind` hooks attached to each container's cgroup enforce use case B's egress policy. Only allow-listed containers may connect to a given network or port, and no container may bind the host's service ports. The verdict is returned on the socket operation, so no packet is ever parsed.
- **Controller (user space, libbpf/CO-RE)**: reads a declarative rule file, populates and updates the maps without reloading programs, pins programs and maps in bpffs, and exports counters and drop logs.

The security property demonstrated is **correct enforcement of a defined policy at lower cost than the conventional mechanism**: every packet or socket operation matching a rule class is blocked, all other traffic passes, and the XDP layer sustains a higher drop rate per CPU than nftables applying the same rules. The firewall is also a target for the threat model of Section 5. Its policy lives in maps, so the map-tamper attack of He et al. [5] applies to it directly, and the privilege audit is part of the evaluation. Figure 2 shows the whole flow.

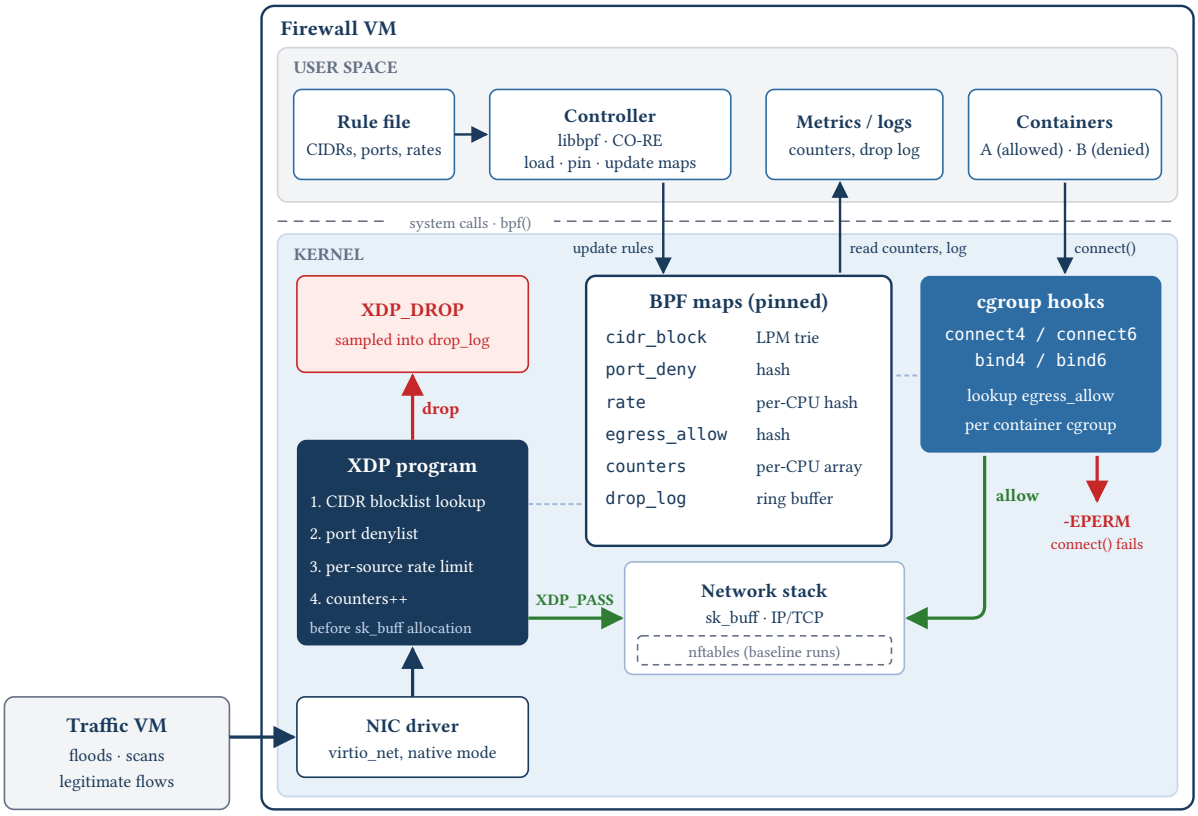


Figure 2. Prototype flow. Inbound packets meet the XDP program in the driver and are dropped or passed before an `sk_buff` exists; container `connect()`/`bind()` calls meet the cgroup hooks. Both read policy from pinned maps that the controller updates at runtime and that feed counters and the drop log back to user space. In baseline runs the XDP program is detached and nftables applies the same rules in the stack.

7.2 Metrics

Table 3. Evaluation criteria for Assignment #2.

Metric	How it is measured	Target / baseline
Enforcement correctness	Positive and negative test cases per rule class (blocked CIDR, denied port, rate-limited source, container egress deny/allow, host-port bind)	100% of cases give the expected verdict, at XDP and at the cgroup hooks
Packet rate	Maximum drop rate (Mpps) with one core, minimum-size packets from a traffic generator, XDP vs. nftables with equivalent rules	XDP above nftables; ratio reported
Latency	Added round-trip latency for allowed traffic (request/response benchmark); time from controller rule update to enforcement	Allowed traffic: within run-to-run noise of no filter; rule update under 1 s
CPU overhead	Softirq and total CPU at fixed attack rates, XDP vs. nftables vs. no filter	Lower CPU than nftables at every rate
Memory / map usage	`bpftool map` sizes for 1k, 10k and 100k CIDR rules; lookup cost vs. rule count	Bounded, linear in rules; reported per rule
False positives / event loss	Legitimate flows dropped while a flood runs; ring-buffer drop counter for sampled logs	Zero legitimate drops outside rate-limited sources; log loss reported, not hidden
Privileges required	Capabilities needed to load and attach each layer; behaviour when they are removed	`CAP_BPF` + `CAP_NET_ADMIN`, no `CAP_SYS_ADMIN`, confirmed by the audit

7.3 Environment

Isolated QEMU/KVM virtual machines on a private virtual network, with a snapshot before each run. One VM generates traffic, one runs the firewall, and the containers under test run on the firewall VM. There are no public Internet targets, and all floods and scans are sent only between our own VMs. The kernel version is pinned and recorded (a recent LTS with BTF enabled, so CO-RE relocations resolve without a patched kernel), and clang, libbpf, bpftool and nftables versions are recorded with the artefacts. The XDP attach mode is recorded for every run: native (driver) mode on `virtio_net` is the target, and generic (skb) mode is reported separately if native mode is unavailable, since generic mode pays the `sk_buff` cost that use case A is meant to avoid. Load comes from a kernel packet generator for floods and from standard throughput and request/response benchmarks for legitimate traffic, so both filters are measured under the same workload.

8 Ethical and legal considerations

All experiments run in isolated virtual machines under our own control, with snapshots taken before each run. Nothing touches production systems, third-party infrastructure, or the public Internet. The attacks used in Assignment #2 (packet floods, port scans, disallowed container connections, and the map-tamper and over-privileged-container scenarios of Section 5) are generated only between VMs and containers we launched ourselves. The firewall inspects only synthetic traffic between our own machines, so no user data enters the counters or logs. The privilege-audit results of the metrics table are the ethical payload of the exercise: if the firewall needs only `CAP_BPF` and `CAP_NET_ADMIN`, the deployment story is one of **less** privilege than the alternatives it replaces, not more.

9 Conclusion

The comparison in this report shows one substrate under four systems, and three findings fall out of it. First, the hook **is** the security decision: the same verified bytecode enforces packet-level filtering at XDP, workload-level policy at the socket layer, and reference monitoring at the BPF LSM, and the two use cases show one question (split-second triage of junk traffic vs. identity-aware control) selecting different hooks for reasons that are economic, not technical. Second, the production and research systems are the same architecture at different points of one design space, and what actually separates them is not capability but policy authorship: who defines identity and what happens when the control plane is compromised. Third, and least settled: the verifier that makes all of this safe is itself a growing, hand-written trusted computing base, one that the literature argues cannot scale as helpers and program types multiply [4], and the cross-container attacks of [5] show that a shared kernel with tracing attached is a shared attack surface regardless of what the verifier proved about the programs.

The main open challenge that follows is measurable safety: neither production system publishes overhead comparable to the research benchmarks, and the deployments surveyed rarely state the privileges they add. The prototype of Assignment #2 addresses that gap at the smallest useful scale: a two-layer firewall, XDP for packets and cgroup/socket hooks for workloads, evaluated for enforcement correctness, cost against nftables and required privilege against the threat model of Section 5. The roadmap below sets out how it will be built and validated.

10 Implementation roadmap for Assignment #2

Table 4. Milestones, dependencies and validation. Weeks count from 2 November 2026 to the Assignment #2 deadline of 6 December 2026. Week 5 (from 30 November) is left free for the Assignment #2 report and packaging.

#	Milestone and deliverable	Depends on	Validation	Week
M1	Minimal XDP program: static CIDR drop with per-CPU counters	-	`bpftool prog` shows it attached; counters match packets sent by the generator	1
M2	User-space controller: rule file → LPM trie, port and rate maps; cgroup `connect/bind` hooks; ring-buffer drop log	M1	Rules added and removed at runtime take effect without reload	1–2
M3	Policy test suite	M2	All positive and negative cases of the metrics table pass at both layers	2
M4	Benchmark harness and nftables baseline	M3	Packet-rate, CPU, latency and map-size runs repeated and scripted	3–4
M5	Threat-model validation: privilege audit and map-tamper test	M3	Loads with `CAP_BPF` + `CAP_NET_ADMIN` only; tampering from a second process is attempted and the result documented	4
M6	Reproducibility artefacts and README	M4, M5	Fresh VM from the README reproduces one full run	4

Bibliography

[1] The Linux Kernel Documentation, "BPF Documentation." [Online]. Available: https://docs.kernel.org/bpf/

[2] T. Høiland-Jørgensen *et al.*, "The eXpress Data Path: Fast Programmable Packet Processing in the Operating System Kernel," in *Proceedings of the 14th International Conference on Emerging Networking EXperiments and Technologies (CoNEXT '18)*, ACM, 2018, pp. 54–66. doi: 10.1145/3281411.3281443.

[3] The Linux Kernel Documentation, "LSM BPF Programs." [Online]. Available: https://docs.kernel.org/bpf/prog_lsm.html

[4] J. Jia, R. Sahu, A. Oswald, D. Williams, M. V. Le, and T. Xu, "Kernel Extension Verification is Untenable," in *Proceedings of the 19th Workshop on Hot Topics in Operating Systems (HotOS '23)*, ACM, 2023, pp. 150–157. doi: 10.1145/3593856.3595892.

[5] Y. He *et al.*, "Cross Container Attacks: The Bewildered eBPF on Clouds," in *32nd USENIX Security Symposium (USENIX Security 23)*, USENIX Association, 2023. [Online]. Available: https://www.usenix.org/conference/usenixsecurity23/presentation/he

[6] Cilium Authors, "BPF and XDP Reference Guide." [Online]. Available: https://docs.cilium.io/en/stable/reference-guides/bpf/

[7] The Linux Kernel Documentation, "eBPF Verifier." [Online]. Available: https://docs.kernel.org/bpf/verifier.html

[8] E. Gershuni *et al.*, "Simple and Precise Static Analysis of Untrusted Linux Kernel Extensions," in *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '19)*, ACM, 2019, pp. 1069–1084. doi: 10.1145/3314221.3314590.

[9] The Linux Kernel Documentation, "BPF Ring Buffer." [Online]. Available: https://docs.kernel.org/bpf/ringbuf.html

[10] libbpf Authors, "libbpf-bootstrap: Scaffolding for BPF Application Development with libbpf and BPF CO-RE." [Online]. Available: https://github.com/libbpf/libbpf-bootstrap

[11] Linux man-pages project, "capabilities(7) — Overview of Linux Capabilities." [Online]. Available: https://man7.org/linux/man-pages/man7/capabilities.7.html

[12] The Linux Kernel Documentation, "Documentation for /proc/sys/kernel/(unprivileged_bpf_disabled)." [Online]. Available: https://docs.kernel.org/admin-guide/sysctl/kernel.html

[13] J. Corbet, "Finer-grained BPF Tokens." [Online]. Available: https://lwn.net/Articles/947173/

[14] W. Findlay, D. Barrera, and A. Somayaji, "BPFContain: Fixing the Soft Underbelly of Container Security." [Online]. Available: https://arxiv.org/abs/2102.06972

[15] Cilium Authors, "Identity-Based Security." [Online]. Available: https://docs.cilium.io/en/stable/security/network/identity/

[16] Cilium Authors, "Envoy (L7 Proxy)." [Online]. Available: https://docs.cilium.io/en/stable/security/network/proxy/envoy/

[17] Cilium Authors, "Tetragon: eBPF-based Security Observability and Runtime Enforcement." [Online]. Available: https://tetragon.io/docs/

[18] W. Findlay, A. Somayaji, and D. Barrera, "bpfbox: Simple Precise Process Confinement with eBPF," in *Proceedings of the 2020 ACM SIGSAC Conference on Cloud Computing Security Workshop (CCSW '20)*, ACM, 2020. doi: 10.1145/3411495.3421358.